```
  p ← Pair(element,
priority)                                                    #1
  pairs.append(p)
                                      #2
  bubbleUp(pairs, |pairs| – 1)
                                                #3
```

#1 Create a new pair holding our new element and priority
#2 Add the newly created pair at the end of our heap's array, incrementing the array's size
#3 After adding the new element, we need to make sure that heap's properties are reinstated

The first two steps in listing 2.6 are self-explanatory: we are just performing some maintenance on our data model, creating a pair from the two arguments and appending it to the tail of our array. Depending on the programming language and the type of container used for pairs, we might have to manually resize it (statically dimensioned arrays) or just add the element (dynamic arrays or lists)[11].

The last step is needed because the new pair might violate the heap's properties we defined in section 2.5.3. In particular, its priority could be higher than its parent's. To reinstate heap properties, we then need to "bubble it up" toward the root of the heap, till we find the right spot in the tree structure.

Insert operations, just like delete or update, or even heap's construction, will have to make sure the heaps properties holds on completion.